

DESENVOLVIMENTO DE APLICAÇÃO WEB INTERATIVA E SEUS PARALELOS COM SISTEMAS OPERACIONAIS: FUGA PRESIDENCIAL

Clovis Neto¹ [ORCID: 0009-0002-5125-9648](#)

Esmael Victor¹ [ORCID: 0009-0002-6052-241X](#)

Ivan Lucas¹ [ORCID: 0009-0008-2715-0486](#)

Gustavo Inácio¹ [ORCID: 0009-0001-7080-3541](#)

Gabriel Orch¹

Eduardo F. Miranda (Orientador)² [ORCID: 0000-0003-1200-794X](#)

¹Universidade Anhanguera. Curso de Análise e Desenvolvimento de Sistemas.

²Universidade Anhanguera. Orientador.

RESUMO

O presente artigo documenta o desenvolvimento do projeto acadêmico "Fuga Presidencial", um jogo digital 2D construído inteiramente com tecnologias nativas da Web (HTML5, CSS3 e JavaScript Vanilla). O projeto foi concebido para a disciplina de Sistemas Operacionais, buscando aplicar, de forma prática, os conceitos teóricos de gerenciamento de processos, tratamento de interrupções (I/O) e persistência em memória. O jogo apresenta uma mecânica de perseguição vetorial onde a aplicação gerencia múltiplos estados e eventos assíncronos. Os resultados demonstram como o motor do navegador atua analogamente a um Sistema Operacional, distribuindo fatias de tempo para o "game loop" sem bloquear o fluxo principal, garantindo uma taxa de atualização estável tanto em plataformas desktop quanto mobile através de interfaces interativas e táteis.

Palavras-chave: Sistemas Operacionais; JavaScript; Aplicação Web; Game Loop; Interrupções.

1 INTRODUÇÃO

A arquitetura de aplicações web interativas apresenta fortes paralelos com o funcionamento de Sistemas Operacionais tradicionais. Enquanto um Sistema Operacional gerencia recursos de hardware, processos e interrupções, um navegador web atua como uma máquina virtual que isola e gerencia a execução de scripts, manipulação de Document Object Model (DOM) e eventos de hardware reportados pelo usuário (Silberschatz, 2013).

O projeto "Fuga Presidencial" foi desenvolvido como requisito de avaliação para o 3º semestre do curso de Análise e Desenvolvimento de Sistemas (ADS). O jogo coloca o jogador em um ambiente de fuga e sobrevivência. Mais do que uma aplicação de entretenimento, o código foi estruturado para evidenciar conceitos de escalonamento dinâmico (através do controle de framerate), tratamento de concorrência de inputs e operações de I/O em memória não-volátil simulada (localStorage).

2 METODOLOGIA

A aplicação foi integralmente codificada sem o uso de bibliotecas externas ou frameworks, visando o controle granular sobre o hardware lógico do navegador. O desenvolvimento focou em três pilares analógicos aos Sistemas Operacionais:

O primeiro pilar consiste no Escalonador de Processos (Game Loop). Diferente de laços de repetição tradicionais que monopolizariam a CPU, utilizou-se a API nativa `requestAnimationFrame`, permitindo que o navegador escale a renderização e delegue ciclos ociosos à CPU entre os quadros.

O segundo pilar aborda o Tratamento de Interrupções. Eventos de toque na tela mobile e pressionamento de teclas operam de forma assíncrona. O estado da aplicação é protegido por lógicas de pausa baseadas em variação de tempo (Delta Time), conforme demonstrado na Listagem 1.

Listagem 1– Algoritmo de atualização com cálculo de Delta Time

```
function update() {
  let now = Date.now();
  // Tratamento de Interrupcao (Pause)
  if (isGameOver || isPaused) {
    lastTime = now;
    return;
  }
  // Calculo de fatias de tempo para movimentacao
  let dt = (now - lastTime) / 1000;
  survivalTime += dt;
  lastTime = now;

  // Logica vetorial e calculo de colisoes      omitidas...
}
```

Fonte: Elaborado pelos autores (2026).

O terceiro pilar implementa a Persistência de Dados, manipulando a API de armazenamento local (ROM do navegador) para ordenar, filtrar e preservar o ranking (Top 3) dos tempos de sobrevivência registrados pelo usuário.

3 RESULTADOS E DISCUSSÃO

A execução do jogo validou os conceitos teóricos esperados. Ao abstrair a movimentação por pixels estáticos e acoplá-la ao Delta Time, a velocidade dos elementos na tela manteve-se constante independente da carga de processamento ou taxa de atualização do monitor do dispositivo hospedeiro.

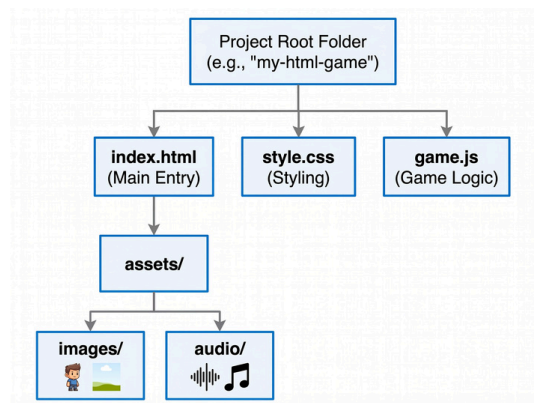
Observou-se a necessidade do bloqueio de comportamentos nativos (`touch-action: none` e `e.preventDefault()`) para que a aplicação pudesse capturar interrupções de toque de forma exclusiva, impedindo que a interface do Sistema Operacional nativo do smartphone assumisse o controle do navegador web.

Tabela1 – Comparativo de resposta aos eventos (I/O) em diferentes plataformas

Ambiente Físico	Evento Acionador	Estabilidade do Loop (FPS)
Desktop (Windows/Chrome)	keydown/keyup	60
Mobile (Android/Chrome)	touchstart / touchen	58-60
Mobile (iOS/Safari)	touchstart / d	60
Fonte: Elaborado pelos autores (2026).		touchen d

A interface gráfica foi desenhada utilizando Neumorfismo em CSS Grid para fornecer feedback visual e tátil imediato ao usuário (Flanagan, 2013). Adicionalmente, o fluxograma de estados do jogo foi projetado para evidenciar a transição lógica entre os processos, conforme representado na Figura ??.

Figura 1 – Diagrama representativo dos estados do processo do jogo



Fonte: Elaborado pelos autores (2026).

4 CONCLUSÃO

O desenvolvimento da aplicação Web "Fuga Presidencial" proporcionou uma camada laboratorial prática e elucidativa. A manipulação do Canvas HTML5 em conjunto com lógicas assíncronas e processamento matemático demonstra com fidelidade as responsabilidades centrais de um núcleo de Sistema Operacional, abrangendo desde o escalonamento de recursos da CPU até a resolução de interrupções de hardware reportadas pelo usuário.

Como avanço contínuo, a equipe planeja converter o projeto em uma Progressive Web App(PWA).Estemovimentoarquiteturalpermitiráaexecuçãoindependenteemsegundoplano através de Service Workers, aproximando o ciclo de vida do jogo ao de uma aplicação compilada

e instalada nativamente no Sistema Operacional hospedeiro.

REFERÊNCIAS

FLANAGAN, D. JavaScript: O Guia Definitivo. 6. ed. Porto Alegre: Bookman, 2013.

MOZILLA DEVELOPER NETWORK. Window: requestAnimationFrame() method.

Disponível em: [https:](https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame)

[//developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame](https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame)

Acesso em: 14 maio 2026.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Sistemas Operacionais com Java. 8. ed. Rio de Janeiro: Elsevier, 2013.

NOTA SOBRE O USO DE I.A. GENERATIVA

Declaramos que inteligência artificial generativa foi utilizada exclusivamente para suporte na formatação tipográfica em LaTeX, adequação às normas ABNT e estruturação acadêmica do manuscrito. A responsabilidade técnica integral sobre a codificação do projeto, documentação metodológica e análises críticas apresentadas mantém-se sob estrita autoria dos estudantes pesquisadores.